SYSTEM DESIGN

SCL PROPULSOR

MECHTRON 4TB6A

Group 11

Winter 2025

January 10, 2025

Pesara Amarasekera, Alexandru Antohe, Nicholas Petrunti,
Daniel Lupas, Samuel Khzym

Table of Contents

Roadmap 4
Introduction 4
Purpose 4
Scope 4
Context Diagram 5
Definitions 5
Hardware 5
    Display Monitor 6
    Safety board - STM32H735G 6
    Vehicle 8
    Arduino 11
    Dyno 11
    Electrical Schematics 12
    Bill of Materials 12
Monitored and Controlled Variables 13
System Constants 14
Calibratable Constants 14
Subsystem and Module Breakdown 15
  Subsystem and Module Diagrams 16
  Subsystem Decomposition 16
  Module Decomposition 16
Modules 17
  M1 – Visualizer 17
    Inputs 17
    Outputs 17
    Description 17
    Initialization 17
    Associated Requirements 18
    Module Breakdown 18
    Methods 19
    Uses Hierarchy 23
  M2 – Environmental Simulator 23

Inputs 23
Outputs 24
Description 24
Hardware Mappings 24
Initialization 24
Associated Requirements 24
Module Breakdown 24
Methods 25
Calculate CRC 27
Calculate RC 27
Uses Hierarchy 29
M3 – Safety Monitor 29
Inputs 29
Outputs 29
Description 29
Hardware Mappings 29
Initialization 30
Associated Requirements 30
Module Breakdown 30
Methods 30
Uses Hierarchy 33
M4 – ACC Algorithm 33
Inputs 33
Outputs 33
Description 33
Hardware Mappings 33
Initialization 33
Associated Requirements 34
Module Breakdown 34
Methods 34
Uses Hierarchy 37
M5 – Power Command Converter 37
Inputs 37

Outputs 37
Description 37
Hardware Mappings 37
Initialization 37
Associated Requirements 38
Module Breakdown 38
Methods 38
Uses Hierarchy 38
M6 – Vehicle Speed Speedometer 38
Inputs 38
Outputs 39
Description 39
Hardware Mappings 39
Initialization 39
Associated Requirements 39
Module Breakdown 39
Methods 40
Uses Hierarchy 42
M7 – Vehicle Plant Model 42
Inputs 42
Outputs 42
Description 42
Initialization 43
Associated Requirements 43
Module Breakdown 43
Methods 43
Uses Hierarchy 44
Normal Behaviour and Unexpected Event Handling 44
Normal Behaviour 44
Unexpected Event Handling 44
References 45

Roadmap

Revision History

Date	Version	Notes
January 10th, 2025	1.0	Draft System Design
March 19th, 2025	2.0	Update to the document – Figure 13. CAD diagram of the dynamometer reflects current design.
March 23rd,2025	3.0	Added some changes Due to Updates to the Requirements

Introduction

This document provides the System Design for SCL Propulsor, a hardware and software simulator for developing Adaptive Cruise Control algorithms. The overall system is broken down into submodules in this document with requirements and pseudocode provided. The most important designs needed for the subsystems are highlighted independent of implementation specifics.

Purpose

The current development of control algorithms for autonomous vehicles is done in a time consuming, uncontrolled and non scalable manner.
The project aims to develop a comprehensive testing environment for an adaptive cruise control algorithm that integrates Software-in-the-Loop (SIL), Hardware-in-the-Loop (HIL), and a real-time visualizer, using an RC (radio-controlled) vehicle as the platform. The setup will facilitate the modeling and evaluation of autonomous driving algorithms and safety mechanisms in a controlled and scalable manner, using the same principles used in an automotive chassis dynamometer which holds the vehicle in place while allowing the wheels to spin.

Scope

The scope of this project is focused on creating Adaptive Cruise Control algorithms for other engineers to use in the future. As such, the scope does not include creating the chassis or wheels of the vehicle nor adding the safety monitor to a car that drives on everyday streets. In addition, the simulator will create roads with straight lanes only as the RC vehicle cannot turn while on the dyno. Finally, it is assumed that the sensor measurements of the lead vehicle speed and distance are accurate and simulating sensors with noise is out of scope.

Context Diagram

The project setup consists of a display monitor with the computer running the simulator, the safety board microcontroller that is directly connected to the computer, an RC vehicle with an on-board Arduino that is connected to the safety board and a fixed platform dynamometer.

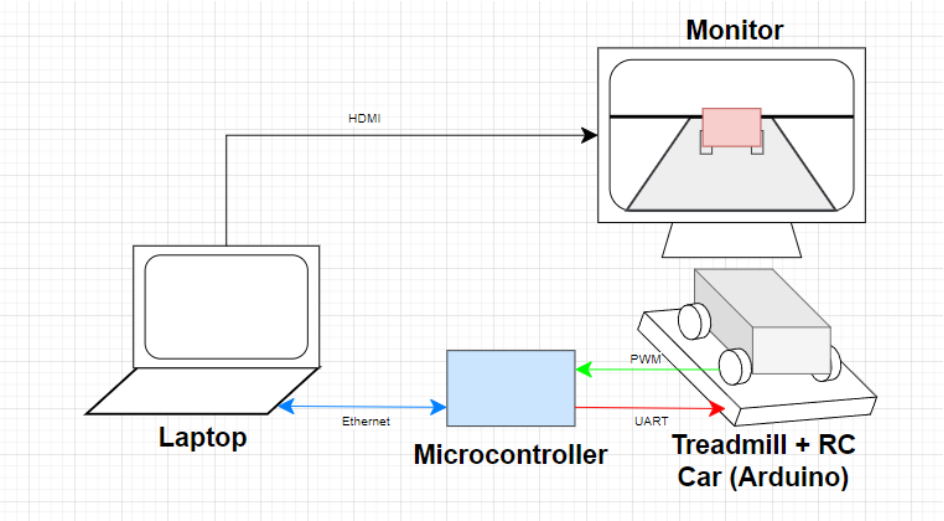


Figure 1. Context Diagram.

Definitions

Definitions and notations that are used throughout the document.

Table 1. Glossary.

Term	Definition
ACC	Adaptive Cruise Control
Steady State	The state where a variable has not deviated more than a threshold (See Steady State Threshold)
MCU	Microcontroller Unit
CRC	Cyclic Redundancy Check
RC	Rolling Count
Ego Vehicle	Vehicle under our control

Hardware

The hardware components of this project consist of a display monitor and computer, the safety board microcontroller, an RC vehicle with an on-board Arduino and a dynamometer.

Display Monitor

The display monitor displays the visualization from the laptop that is running the simulator. The computer is connected to the safety board microcontroller through an ethernet cable.

Safety board - STM32H735G

The safety board that is responsible for monitoring SIL and HIL testing is an STM32H735G Disco [1]. The physical dimensioning of the board is outlined below.

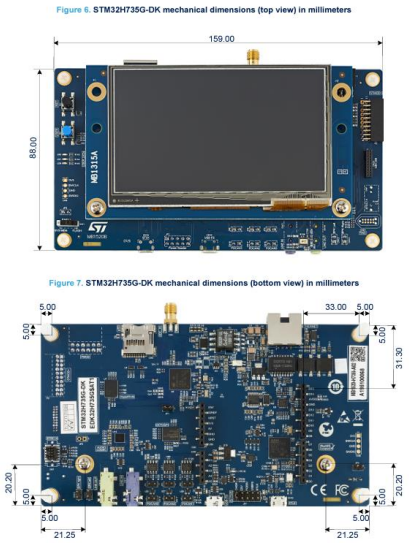


Figure 2. Microcontroller Dimensions (in mm).

To protect the microcontroller from damage and to ease the mounting process, a case was designed to enclose it.

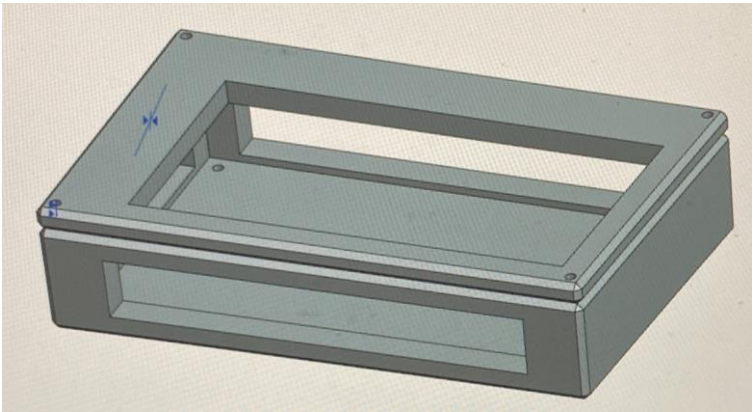


Figure 3. Microcontroller CAD Modelled Case.

The microcontroller has many hardware interfaces available for use. For the scopes of this project the hardware interfaces that will be used include the PWR supply, ethernet, and the

STMod+ connector to facilitate UART serial communication between the microcontroller and the Arduino on-board the RC vehicle.

Figure 3. **Hardware block diagram**

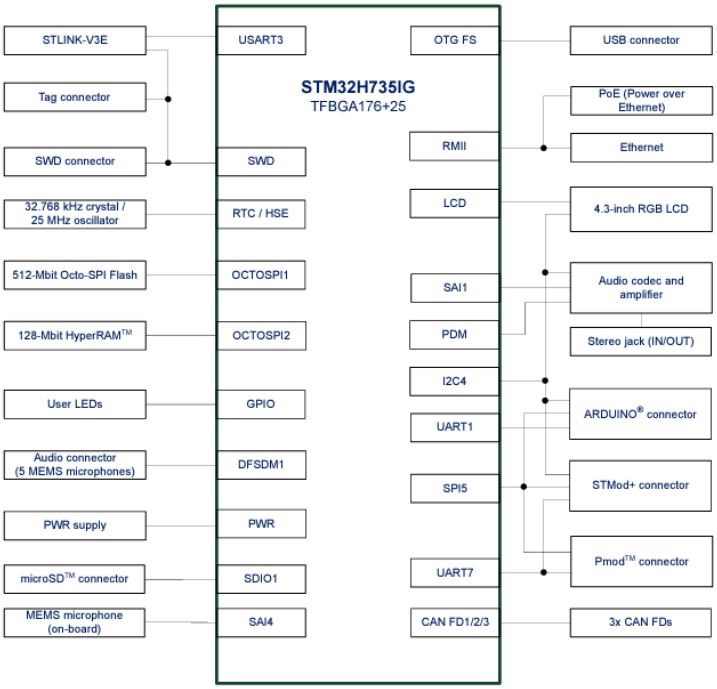
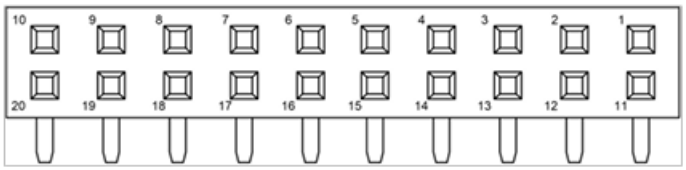


Figure 4. STM32H735G Hardware Interfaces.

Figure 13. **P2 STMod+ connector**



Front view

Table 15. **P2 STMod+ connector pinout**

Pin number	Description	Pin number	Description
1	SPI5_NSS / USART7_CTS (PF6/PF9)	11	INT (PH12)
2	SPI5_MOSI / USART7_TX (PF9/PF7)	12	RESET (PH1)
3	SPI5_MISO / USART7_RX (PF8/PF6)	13	ADC (PA5)
4	SPI5_SCK / USART7_RTS (PF7/PF8)	14	PWM (PD14)
5	GND	15	+5V
6	+5V	16	GND
7	I2C4_SCL (PF14)	17	DFSDM1-DATIN2 (PE7)
8	SPI5_MOSIs (PF11)	18	DFSDM1-CKOUT (PE9)
9	SPI5_MISOs (PH7)	19	DFSDM1-DATIN4 (PE10)
10	I2C4_SDA (PF15)	20	DFSDM1-DATIN6 (PF13)

Figure 5. P2 STMOD+ Connector.

Vehicle

The RC vehicle chosen for this project was the Elegoo Smart Car V4.0 [2].

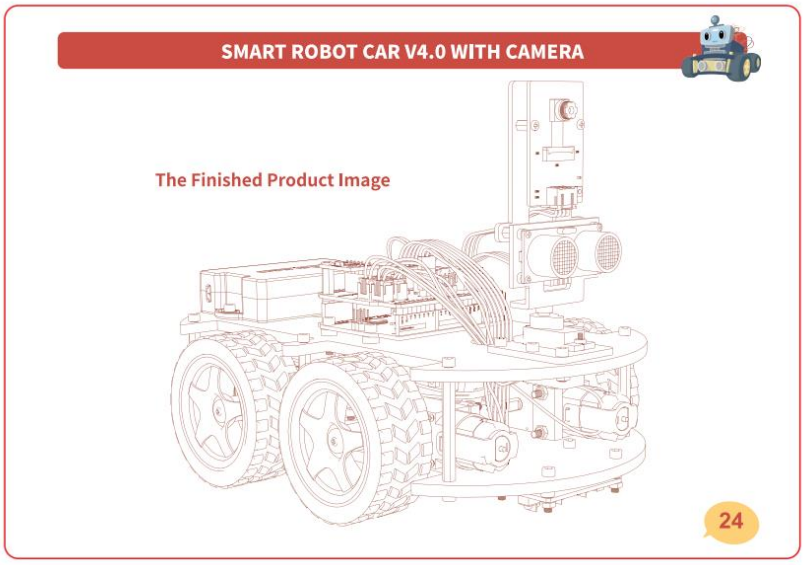


Figure 6. Elegoo RC Vehicle.

The vehicle is equipped with four DC motors, a 5V battery pack, and an on-board Arduino. For this vehicle, the two motors on the left side are powered by the same PWM signal, and the two motors on the right side are powered by the same PWM signal. This is due to the on-board wiring of the h-bridge motor drive [3] that controls the motor's PWM signal.

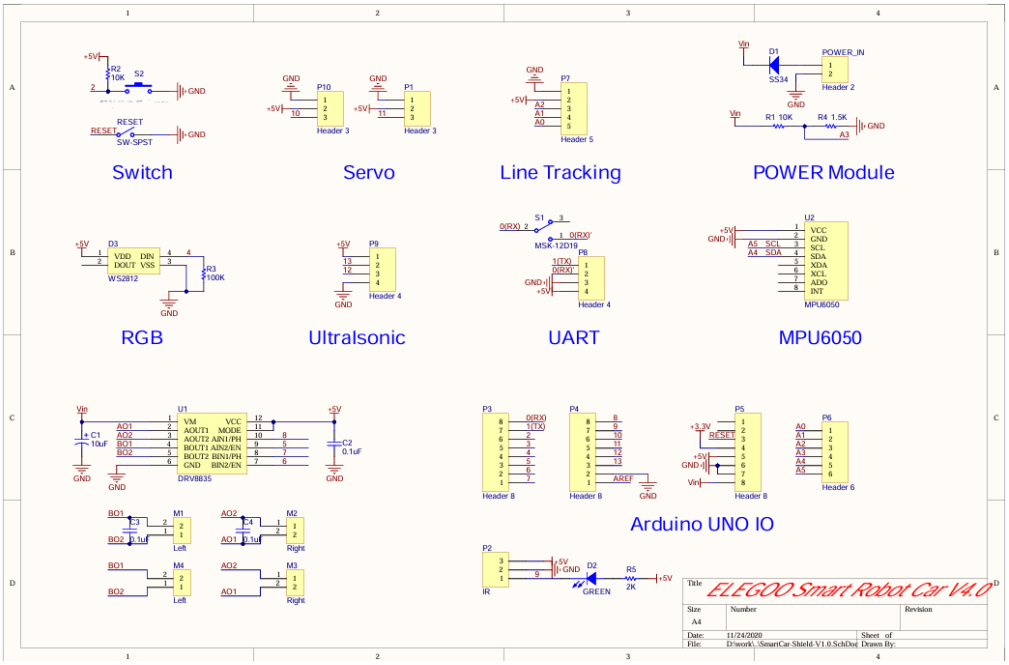


Figure 7. Elegoo Smart Roboto Car V4.0 Schematics.

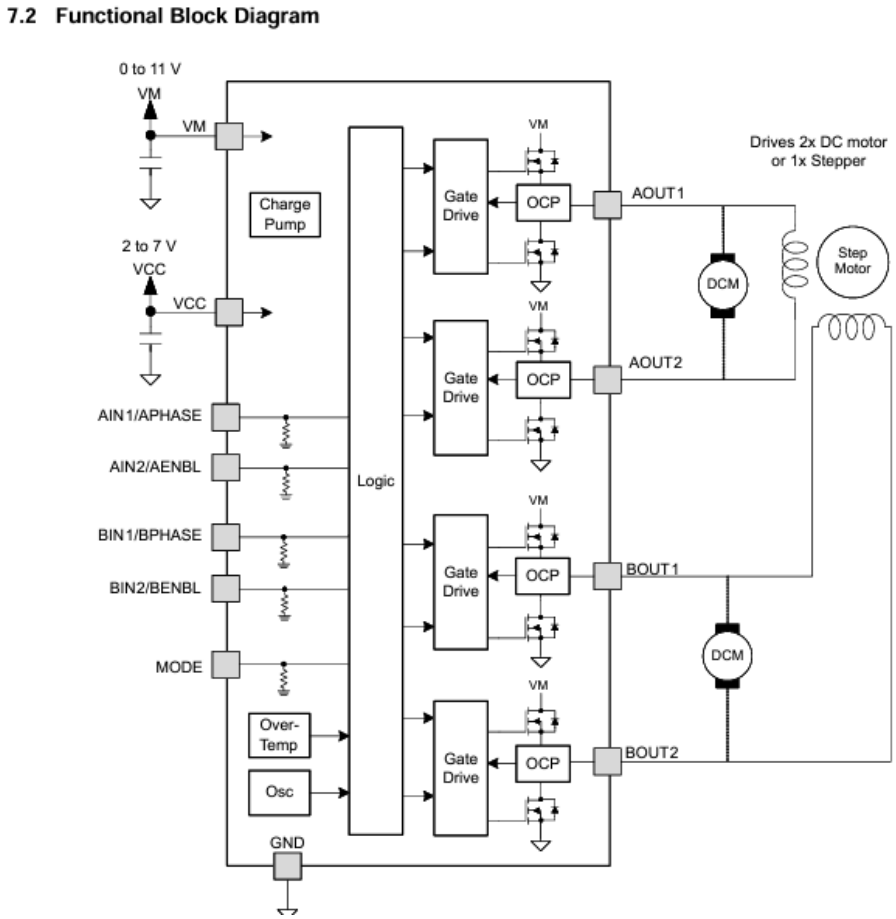


Figure 8. Functional Block Diagram of H-Bridge Driver.

The RC vehicle will have hall effect sensors [4] that will act as rotation sensors. The sensors have a 50% nominal duty cycle at 2 kHz, with the duty cycle increasing or decreasing based on the magnetic field perpendicular to the sensor. These will be mounted on black acrylic plates and will be able to monitor the speed of the wheels.

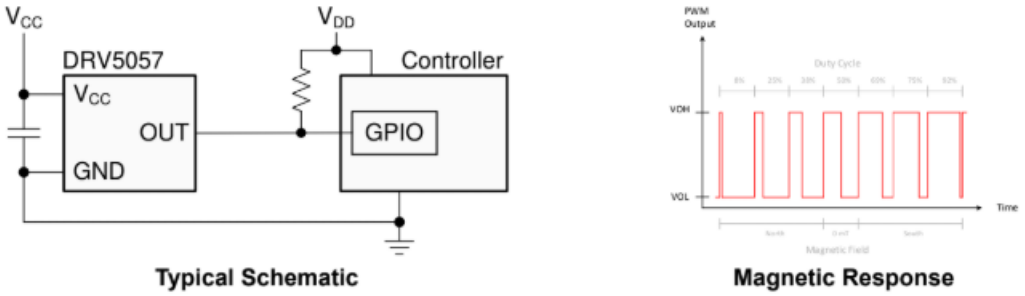


Figure 9. Hall - Effect Sensor Schematic and Magnetic Response.

5 Pin Configuration and Functions

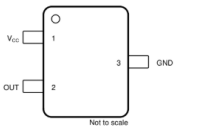


Figure 5-1. DBZ Package 3-Pin SOT-23 Top View

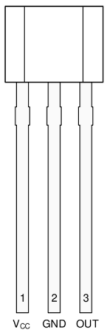


Figure 5-2. LPG Package 3-Pin TO-92 Top View

Pin Functions

PIN			TYPE	DESCRIPTION
NAME	SOT-23	TO-92		
GND	3	2	Ground	Ground reference
OUT	2	3	Output	Analog output
V_{CC}	1	1	Power	Power supply. Connect this pin to a ceramic capacitor to ground with a value of at least 0.01 µF.

Figure 10. Hall-Effect Pins.

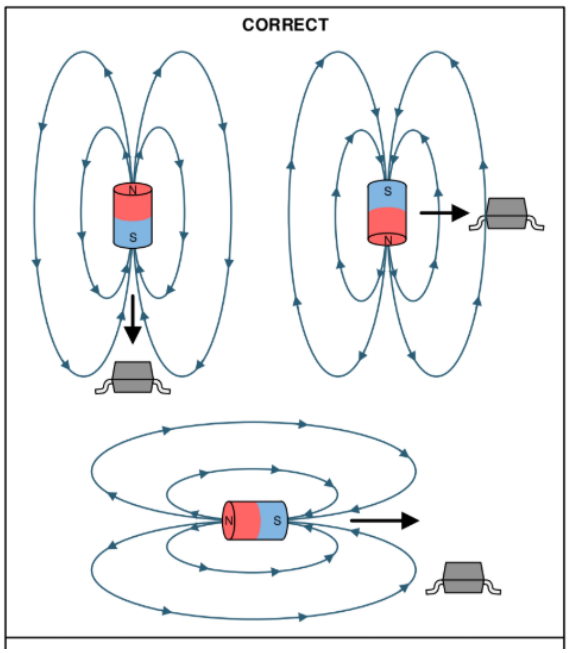


Figure 11. Ideal Mounting of Magnet for Hall Effect Sensor.

7.2 Functional Block Diagram

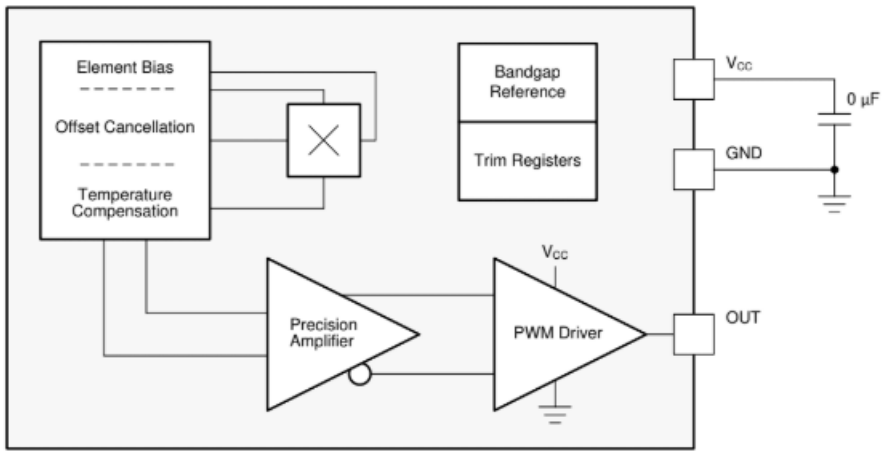


Figure 12. Hall Effect Functional Diagram.

Arduino

The on-board Arduino will be responsible for transmitting the PWM signals to the motors, monitoring the hall-effect sensor duty cycles, and bidirectionally transmitting to the microcontroller through UART.

Dyno

To facilitate the visualization of the RC vehicle moving in accordance with the control algorithms, a fixed dynamometer platform was designed to support the vehicle and to allow the wheels to spin in free rotation.

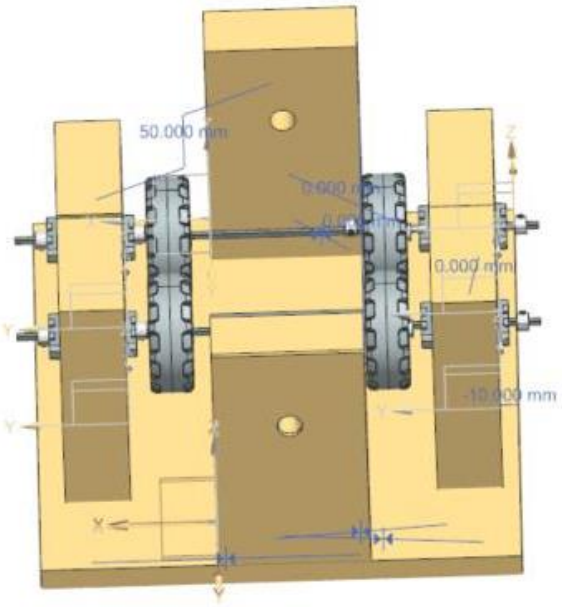


Figure 13. Fixed Dynamometer Platform.

Electrical Schematics

The electrical schematic for the hardware of this project can be seen below.

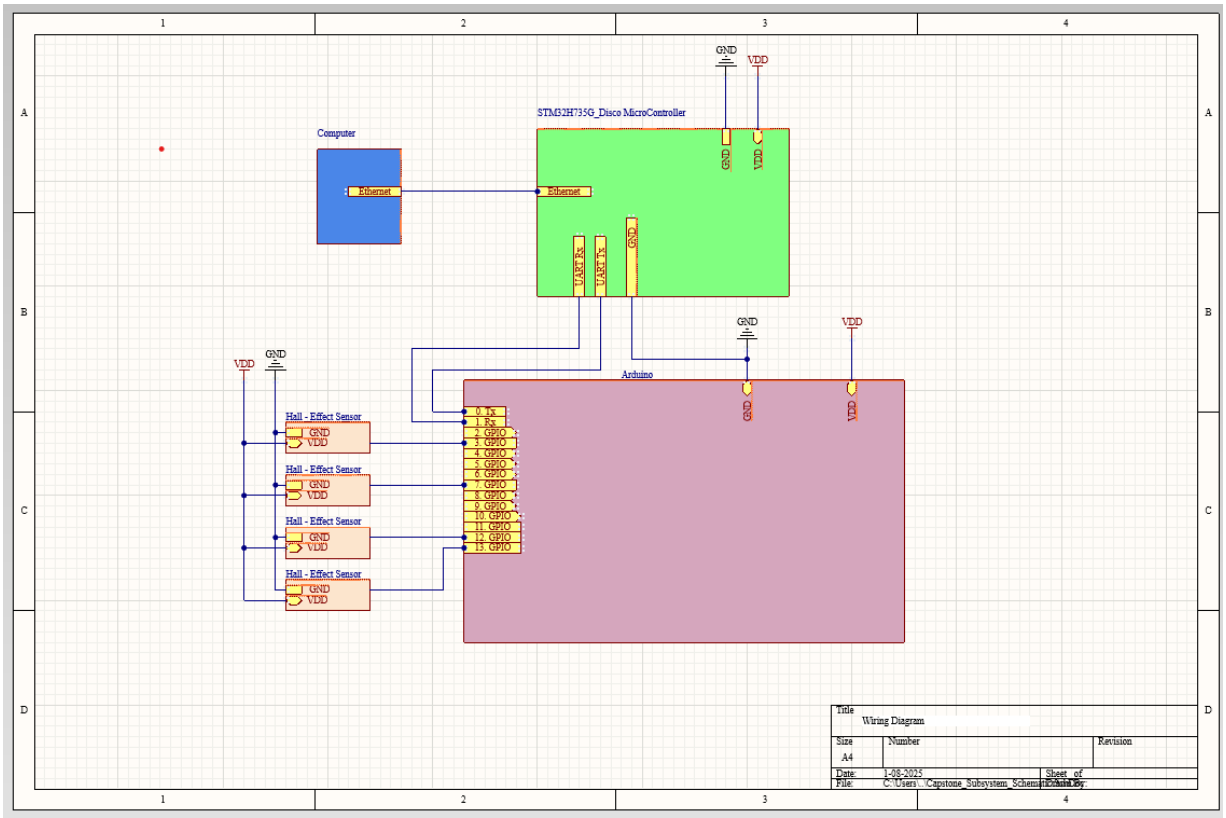


Figure 14. Hardware Electrical Schematic.

Bill of Materials

The bill of materials currently accounts for the components that would need to be purchased to complete hardware design.

Item	Price	Quantity	Online Link
STM32H735G-DK Microcontroller	$133.51	$1.00	https://www.digikey.ca/en/products/detail/stmicroelectronics/STM32H735G-DK/13171216
DRV5057A1QLPGM Hall - Effect Sensor	$3.44	$4.00	https://www.digikey.ca/en/products/detail/texas-instruments/DRV5057A1QLPGM/9860902
Neodymium Magnets	$17.99	1	https://www.amazon.ca/dp/B07W8GLDKY
Smart Robot Car V4.0	$100.78	1	https://us.elegoo.com/products/elegoo-smart-robot-car-kit-v-4-0
Square Drive Shafts 12" (4 Pack)	$13.99	1	https://www.vexrobotics.com/drive-shafts.html#cad
Flat Bearing (10 Pack)	$7.99	1	https://www.vexrobotics.com/v5-bearings.html
Shaft Collars (16 Pack)	$12.99	2	https://www.vexrobotics.com/v5-shaft-collars.html
Traction Wheels (2 Pack)	$12.99	2	https://www.vexrobotics.com/wheels.html#cad
Wood 4x4	$18.99	1	https://www.homedepot.ca/product/micropro-sienna-4-x-4-x-8-pressure-treated-wood-post-suitable-for-ground-contact-/1000790178
Wood 2x4	$3.99	2	https://www.homedepot.ca/product/2-inch-x-4-inch-x-8-ft-spf-select-2btr-grade-lumber/1000112108
	Total: $366.95		

Figure 15. Bill of Materials.

Monitored and Controlled Variables

Table 2. Monitored & Controlled Variables.

Term	Definition
Monitored Variables	
V_E	The velocity of the ego (host) vehicle in $\frac{cm}{s}$.
V_L	The longitudinal velocity of the lead vehicle in $\frac{cm}{s}$.
V_S	The target velocity of the ego vehicle in $\frac{cm}{s}$.
V_{Err}	The absolute difference between set speed and ego velocity; $V_{Err} = \lvert V_S - V_E \rvert$
t_{Err}	The absolute difference between the desired time gap and the actual time gap in seconds; $t_{Err} = \lvert t_{gap} - \frac{D_L}{V_E} \rvert$
a_x	Longitudinal acceleration ($\frac{cm}{s^2}$), the first derivative of ego velocity; $a_x = \frac{dV_E}{dt}$
J_x	Longitudinal jerk, the second derivative of ego velocity ($\frac{cm}{s^3}$); $J_x = \frac{d^2V_E}{dt^2}$
$W_1; W_2$	The ego velocity according to wheel speed sensor 1 and 2 respectively ($\frac{cm}{s}$)
W	The average of W_1 and W_2 ($\frac{cm}{s}$); $W = \frac{W_1+W_2}{2}$
$T_1; T_2$	The ego velocity according to treadmill speed sensor 1 and 2 respectively ($\frac{cm}{s}$)
T	The average of T_1 and T_2 ($\frac{cm}{s}$); T= $\frac{T_1+T_2}{2}$
D_L	The distance from the ego vehicle to the lead vehicle (cm). Distance is relative to the ego vehicle.
$t_{failure}$	The latest timestamp when System failure occurred (seconds)
$t_{request_sim}$	The latest timestamp when a request was sent to the Simulator (seconds)
Controlled Variables	
Act_Req	The actuation request from the microcontroller to the RC car. This is a unitless value that ranges from 0-255 that correlates to torque applied to wheels.

State	The operational state of the ACC. $State = ACTIVE$ when the $V_E \geq 20\frac{cm}{s}$, and $INACTIVE$ otherwise.
$\boldsymbol{V_{FL}, V_{FR}, V_{RL}, V_{RR}}$	Front-left, front-right, rear-left, and rear-right voltages sent to the four DC motors controlling each wheel.
$\boldsymbol{CRC_{Sim}, CRC_{Ard}}$	Cyclic redundancy check output by simulator and Arduino subsystems, sent to microcontroller subsystem
$\boldsymbol{RC_{Sim}, RC_{Ard}}$	Rolling count output by simulator and Arduino subsystems, sent to microcontroller subsystem
$\boldsymbol{FAULT_SPEED}$	Speed fault detection flag

System Constants

Table 3. Constant Variables.

Variable	Definition	Value
$\boldsymbol{V_{Thres}}$	The speed above which ACC is considered active	$20\frac{cm}{s}$
$\boldsymbol{Act_{Min}}$	The minimum allowed value for Act_Req (aside from 0)	100
$\boldsymbol{Act_{Max}}$	The maximum allowed value for Act_Req	255
$\boldsymbol{t_{gap}}$	The ideal time gap	$2\ s$
$\boldsymbol{Dyn_{max}}$	The maximum difference between wheel speeds and treadmill speed.	$5\frac{cm}{s}$
$\boldsymbol{dt_{ACC}}$	The timestep size in the ACC algorithm, simulator, and safety monitor	$10ms$
$\boldsymbol{dt_{Vis}}$	The timestep size in the visualizer	$20ms$
$\boldsymbol{dt_{Ard}}$	The timestep size in the Arduino	$5ms$

Calibratable Constants

Table 4. Calibratable Variables.

Variable	Definition	Default

$\boldsymbol{D_Max}$	The maximum distance that the simulator will consider a lead vehicle present.	$250\ cm$
$\boldsymbol{SS_{Max}}$	The maximum amount of acceptable deviation in a velocity for it to be defined as steady state.	$5\frac{cm}{s}$
$\boldsymbol{V_{ErrMaxSS}}$	The maximum allowable set speed error when in steady state	$5\frac{cm}{s}$
$\boldsymbol{V_{ErrMax}}$	The maximum allowable set speed error when not in state	$15\frac{cm}{s}$
$\boldsymbol{V_{ErrSM}}$	The maximum allowable difference between the set speed and detected speed	$10\frac{cm}{s}$
$\boldsymbol{t_{ErrMaxSS}}$	The maximum allowable time gap error when in steady state	$1\ s$
$\boldsymbol{t_{ErrMax}}$	The maximum allowable time gap error when not in steady state	$3\ s$
$\boldsymbol{a_{max}}$	The maximum allowable longitudinal acceleration	$12\ cm/s^2$
$\boldsymbol{J_{max}}$	The maximum allowable longitudinal jerk	$12\ cm/s^3$
$\boldsymbol{Act_Rate_{Max}}$	The maximum allowable rate of change of actuation request	$\frac{10}{s}$

Subsystem and Module Breakdown

The SCL Propulsor project can be broken down into seven distinct modules between three different software subsystems – the computer, STM microcontroller, and Arduino. There is also the RC Vehicle and Dyno mechanical setup, which is considered a separate mechanical "subsystem". Three of these modules are shared between the SIL and HIL setups, three are unique to the HIL setup, and one is unique to the SIL setup. The relations between these modules and which subsystem they shall be implemented in the SIL/HIL setups are shown in Figure 16 and Figure 17 respectively. Note that monitored and controlled variables distinctions can be different depending on the module, so monitored and controlled variables were assigned based on the role of the quantity in the entire system. Some discretion was taken to decide this.

These modules are divided between the three software subsystems. The computer contains the visualizer and environmental simulator modules (SIL/HIL), the RC vehicle plant model (SIL only), and the ACC algorithm (SIL only). The microcontroller contains the safety monitor and ACC algorithm (HIL only), and the Arduino contains the power command converter and vehicle speed determination modules (HIL only). Interface specifications between modules across

different subsystems are documented in the appropriate module writeups that communicate on the boundaries in the HIL setup.

Subsystem and Module Diagrams

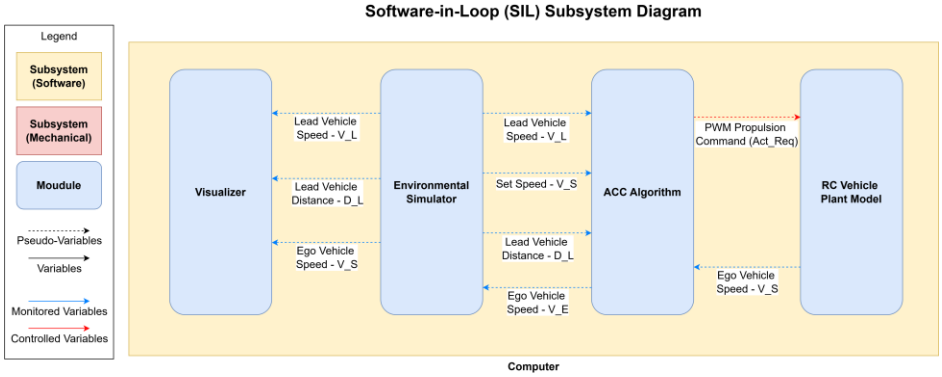


Figure 16: SIL Subsystem Diagram

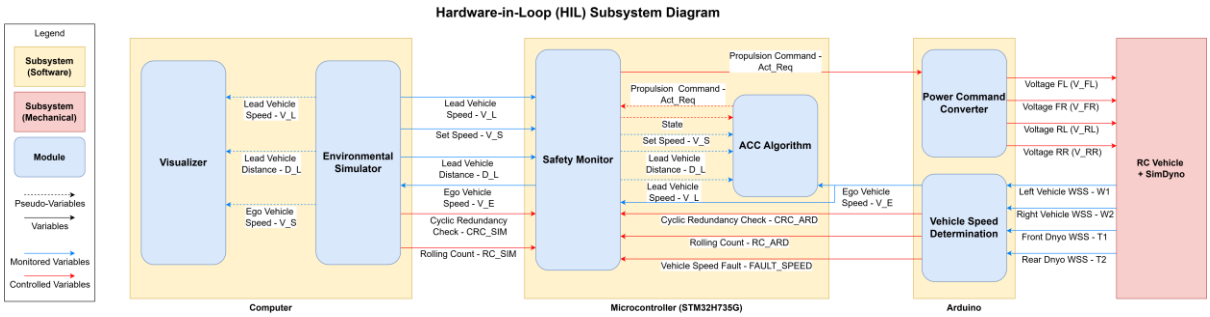


Figure 17: HIL Subsystem Diagram

Subsystem Decomposition

Subsystem ID	Subsystem Name	Base Schedule Rate
S1	COMPUTER	$dt_{ACC} = 10ms$
S2	MICROCONTROLLER	$dt_{ACC} = 10ms$
S3	ARDUINO	$dt_{Ard} = 5ms$

Module Decomposition

Module ID	Module Name	Subsystem (SIL)	Subsystem (HIL)
M1	VISUALIZER	Computer	Computer
M2	ENVIRONMENTAL_SIMULATOR	Computer	Computer
M3	SAFETY_MONITOR	N/A (Unused)	Microcontroller
M4	ACC_ALGORITHM	Computer	Microcontroller
M5	POWER_COMMAND_CONVERTER	N/A (Unused)	Arduino
M6	VEHICLE_SPEED_DETERMINATOR	N/A (Unused)	Arduino
M7	VEHICLE_PLANT_MODEL	Computer	N/A (Unused)

Modules

M1 – Visualizer

Inputs

Lead Vehicle Distance – D_L
Lead Vehicle Speed – V_L
Ego Vehicle Speed – V_E
Ego Vehicle Position – D_E (absolute position in meters)

Outputs

No direct outputs, but produces an animation window of the lead vehicle's relative position and velocity from the perspective of the ego vehicle.

Description

Creates a visual animation of the environment in the perspective of the ego vehicle. The sky shall be represented in blue, the sides of the road in green, and the road in gray. White dashed lane lines shall be drawn on the road and appear to move with respect to the ego vehicle based on its speed. The lead vehicle shall be represented as a red box with black tires. Statistics about the state of the ego and lead vehicle shall be overlaid onto the surface. An example (incomplete) animation frame is shown in Figure 18.

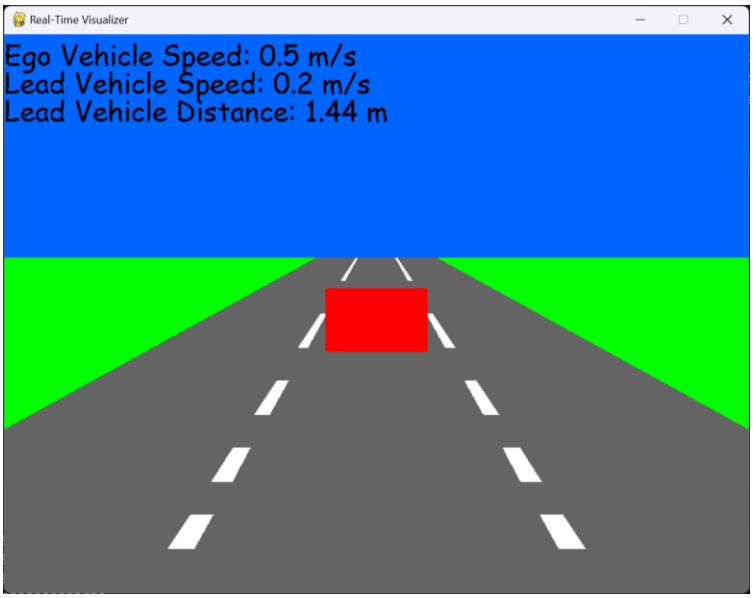


Figure 18: Sample Visualizer Frame

Initialization

Activates upon powering on of system.

Associated Requirements

Visual representation of system state. No requirements were initially written to define behavior.

Software Requirements	N/A
Safety Requirements	N/A
Security Requirements	N/A
Performance Requirements	N//A

Module Breakdown

The visualizer module has two categories of objects it needs to draw – objects that appear static relative to the ego vehicle such as the road shape and the sky, and objects that change depending on the state of the ego vehicle such as the lead vehicle and the dashed lane lines. The largest challenge of this module is the projection of the 3D environment into a 2D space, which is one of the major determining factors behind the structure of the Visualizer and its corresponding method breakdown. The implementation shall be done in Python using the Pygame library due to its rich library of drawing/animation functions.

The UML diagram for the Visualizer class is shown in Figure 19. The initialize method initializes the Pygame window and all of the class attributes. The visualizer has attributes representing physical dimensions such as the maximum length, width, and depth represented by the visualizer, the road width, the lead vehicle dimensions, the lane line dimensions, and lane line spacing (all in metres). To account for the perspective of the ego vehicle, there is also a max distortion parameter which linearly scales the object to be smaller as a function of depth (where max visualizer depth corresponds to the max distortion, and 0 depth corresponds to 0 distortion).

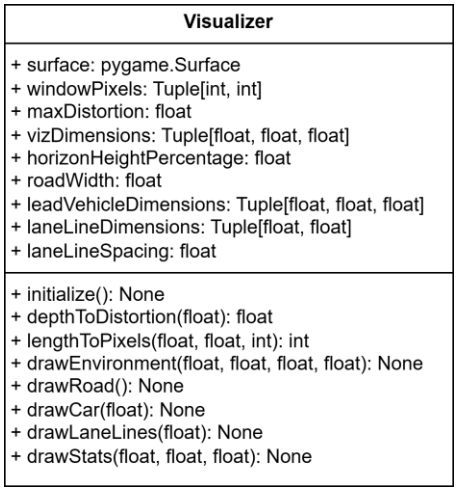


Figure 19: UML Diagram for Visualizer Class (Python)

The depthToDistortion method handles the calculation of the distortion at a given depth, and the lengthToPixels method determines the percentage of the screen the object should take given its proportion of the visualizer dimensions, which determines the number of pixels it should be. These two helper methods are used by the drawEnvironment, drawRoad, and drawCar

methods, which are all invoked (along with drawStats) by the drawEnvironment entry point every visualizer timestep.

Methods

Initialize

Description

Setup animation window and initialize all visualizer parameters.

Scheduling

Called once upon initialization of visualizer subsystem.

Pseudocode

```
void initialize()

      // Initialize window
      this.windowPixels = (800, 600)
      this.surface initialized to size of this.windowPixels

      // Max distortion parameter
      this.maxDistortion = 2.0

      // Max length, height, depth of visualizer (m)
      this.vizDimensions = (0.5, 0.3, 0.2)

      // Horizon params
      this.horizonHeightPercentage = 0.6

      // Lanes params
      this.roadWidth = 0.9

      // lane line params
      this.laneLineDimensions = (0.02, 0.2)
      this.laneLineSpacing = this.laneLineDimensions[0]

      // lead vehicle params
      this.leadVehicleDimensions = (0.2, 0.3, 0.1)
```

Depth To Distortion

Description

Calculates percentage of distortion based on depth (distance from ego vehicle). 0 depth implies distortion ratio of 1, while max depth implies distortion ratio of 1/MAX_DISTORTION.

Scheduling

At the same rate as the module (50Hz).

Pseudocode

```
float depthToDistortion(float depth)
      float slope = (1 / this.maxDistortion - 1) / this.vizDimensions[2]
      return (slope * depth) + 1
```

Design Notes

Equation derived from linear interpolation between (0, 1) and (MAX_DEPTH, MAX_DISTORTION).

Length to Pixels

Description

Converts a physical length into a pixel quantity based on the distance to the ego vehicle (depth).

Scheduling

At the same rate as the module (50Hz).

Pseudocode

```
float lengthToPixels(float distance, float depth, int dimension)
      float distortion = this.depthToDistortion(depth)
      float distortedLength = distance * distortion
      return (int)((distortedLength / this.vizDimensions[dimension]) *
      this.windowPixels[dimension])
```

Design Notes

Dimension 0 indicates width, dimension 1 indicates height.

Draw Environment

Description

Entry point to Visualizer subsystem; dispatches information to each of the individual draw modules.

Scheduling

At the same rate as the module (50Hz).

Pseudocode

```
drawEnvironment(float leadDistance, float leadSpeed, float egoSpeed, float
egoPosition)
      drawRoad()
      drawLaneLines(egoPosition)
      drawCar(leadDistance)
      drawStats(leadDistance, egoSpeed, leadSpeed)
```

Design Notes

N/A

Draw Road

Description

Draws horizon, sides of road, and road onto canvas window.

Scheduling

At the same rate as the module (50Hz).

Pseudocode

```
void drawRoad()

      draw green rectangle on this.surface at (0,0) with height
      this.windowPixels[1] and length this.windowPixels[0]

      draw blue rectangle on this.surface at (0,0) with height
      (int)(this.horizonHeightPercentage * this.windowPixels[1]) and length
      this.windowPixels[0]

      // determine pixel lengths at close and far edge of road
      int closeRoadWidthPx = this.lengthToPixels(this.roadWidth, 0, 0)
      int farRoadWidthPx = this.lengthToPixels(this.roadWidth,
      this.vizDimensions[2], 0)

      // determine Y-values (height on canvas)
      int closeY = 0
      int farY = this.windowPixels[1] * this.horizonHeightPercentage

      // draw road polygon
      draw gray polygon on this.surface with corners at
      (this.windowPixels[0]/2 – closeRoadWidthPx/2, closeY),
      (this.windowPixels[0]/2 + closeRoadWidthPx/2, closeY),
      (this.windowPixels[0]/2 + farRoadWidthPx/2, farY),
      (this.windowPixels[0]/2 - farRoadWidthPx/2, farY)
```

Design Notes

All these functions will be using the Pygame library in Python, which has in-built functions for drawing simple shapes and polygons of different colors. This will be used for all subsequent module pseudocode.

Draw Car

Description

Draws box representing lead vehicle with a certain distance from the ego vehicle.

Scheduling

At the same rate as the module (50Hz).

Pseudocode

```
void drawCar(float leadDistance)

      // determine close and far pixel widths of lead vehicle
      int closeWidth = this.lengthToPixels(this.leadVehicleDimensions[0],
      leadDistance, 0)
      int farWidth = this.lengthToPixels(this.leadVehicleDimensions[0],
      leadDistance + this.leadVehicleDimensions[2], 0)

      // determine close pixel height of lead vehicle
      int closeHeight = this.lengthToPixels(this.leadVehicleDimensions[1],
      leadDistance, 1)

      // determine base y-dimension (height position on canvas)
      int baseY = (int)(horizonHeightPercentage * this.windowPixels[1] *
      (leadDistance / this.vizDimensions[2]))

      // draw red lead vehicle box
      draw red rectangle on this.surface with corners
      (this.windowPixels[0]/2 – closeWidth/2, closeY),
      (this.windowPixels[0]/2 + closeWidth/2, closeY),
      (this.windowPixels[0]/2 + closeWidth/2, closeY + closeHeight),
      (this.windowPixels[0]/2 - closeWidth/2, closeY + closeHeight)
```

Design Notes

This pseudocode describes the front box. The full design with the rest of the shape shall be elaborated on in the final report.

Draw Lane Lines

Description

Draws dashed white lane lines to the left and right of the ego and lead vehicle based on the ego vehicle's absolute position in the environment.

Scheduling

At the same rate as the module (50Hz).

Pseudocode

```
void drawLaneLines(float egoPosition)
      offset = egoPosition % (this.laneLineDimensions[1] +
      this.laneLineSpacing)

      closeDepth = offset
      farDepth = offset + this.laneLineDimensions[1]
      while (farDepth <= this.vizDimensions[2])
```

```
        draw left and right lane line offset from center and determine
        corresponding close width and far width to draw white polygon on
        this.surface
```

Design Notes

The algorithm to shift the perspective off-center is not complex but long to write down, so it will be further elaborated upon in the final report. The operating principle behind the lane line animation is that the offset will change more quickly as the lead vehicle speed increases, resulting in the illusion that the lane lines are moving toward the screen more quickly.

Draw Stats

Description

Displays environment statistics (lead vehicle distance/speed, ego speed) on window as overlay.

Scheduling

At the same rate as the module (50Hz).

Pseudocode

```
void drawStats(float egoSpeed, float leadSpeed, float leadDistance)
      render “Ego Vehicle Speed: ” + (string)(egoSpeed) + “m/s” in black to
      this.surface at (0, 0)
      render “Lead Vehicle Speed: ” + (string)(leadSpeed) + “m/s” in black to
      this.surface at (0, 20)
      render “Lead Vehicle Distance: ” + (string)(leadDistance) + “m” in
      black to this.surface at (0, 40)
```

Design Notes

N/A

Uses Hierarchy

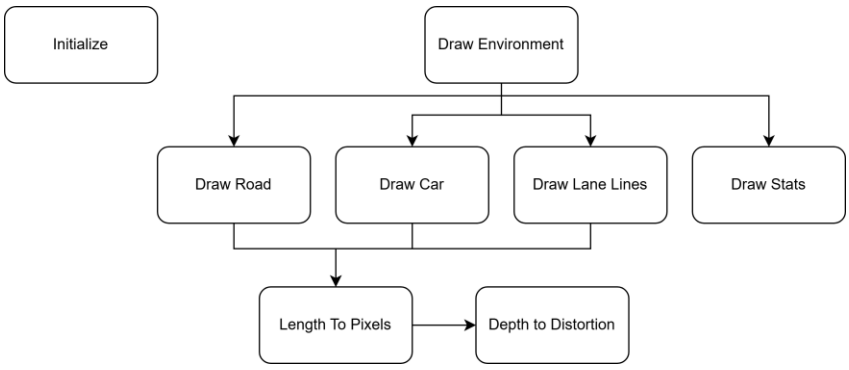


Figure 20: Uses Hierarchy for Visualizer Module

M2 – Environmental Simulator

Inputs

Ego Vehicle Speed – V_E

Outputs

Lead Vehicle Distance – D_L
Lead Vehicle Speed – V_L
Ego Vehicle Speed – V_E
Set Speed – V_s
Cyclic Redundancy Check (Simulator) – *CRC_SIM*
Rolling Count (Simulator) – *RC_SIM*

Description

Creates a realistic speed profile of the lead vehicle based on a user-selected drive cycle. Calculates the lead distance based on the relative velocities of the ego and lead vehicles. Also assigns the set speed based on user settings, which may change during drive cycles. All this information is passed to the Visualizer. CRC and RC checks are sent to the Safety Monitor to ensure data correctness.

Hardware Mappings

Inputs

Name	Input Protocol/Format	Conversion	Variable Datatype
V_E	Ethernet	N/A	Float32

Outputs

Name	Output Protocol/Format	Conversion	Variable Datatype
V_S	Ethernet	Zephyr Eth library	Float32
V_L	Ethernet	Zephyr Eth library	Float32
D_L	Ethernet	Zephyr Eth library	Float32
CRC_SIM	Ethernet	Zephyr Eth library	Uint8
RC_SIM	Ethernet	Zephyr Eth library	Uint8

Initialization

Activates upon powering on of system.

Associated Requirements

Software Requirements	Sim_1, Sim_2
Safety Requirements	N/A
Security Requirements	N/A
Performance Requirements	Perf_1

Module Breakdown

This module handles the processing of drive cycle csv files, the generation of the lead vehicle drive cycle, and the bookkeeping for the absolute positions of the ego and lead vehicle with respect to the absolute reference frame. This can be implemented as a Python class, so this

module can be structured with the class attributes and methods in the UML diagram in Figure 21.

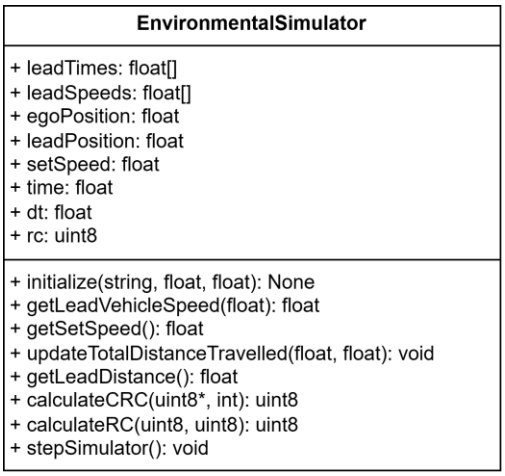


Figure 21: UML Diagram for EnvironmentalSimulator Class

Methods

Initialize

Description

Initializes class member variables such as set speed and constructs drive cycle based on user-provided csv.

Scheduling

Once upon system initialization.

Pseudocode

`void initialize(string driveCycleCsvPath, float setSpeed, float initLeadDistance)`

```
    // parse csv to get time and speed columns
    float[][] driveCycle = readCsv(driveCycleCsvPath)
    float[] times = driveCycle[:,0]
    float[] speeds = driveCycle[:,1]

    // set member variables for lead vehicle data and set speed
    this.leadTimes = times
    this.leadSpeeds = speeds
    this.setSpeed = setSpeed

    // initialize positions of vehicles in environment
    this.egoPosition = 0
    this.leadPosition = initLeadDistance
    this.time = 0
```

```
    this.dt = 0.01
    this.rc = 0
```

Design Notes

Since this module is written in Python, it is possible to use libraries for csv reading and interpolator construction (ie. `numpy.genfromtxt()`).

Get Lead Vehicle Speed

Description

This module takes in a time t and returns a corresponding speed V_L representing the speed of the lead vehicle at that time.

Scheduling

Every timestep of the module (100Hz).

Pseudocode

```
float getLeadVehicleSpeed(float time)
    return interpolate(this.times, this.speeds, time)
```

Design Notes

Since this module is written in Python, it is possible to use libraries for interpolator construction (ie. `scipy.interpolate.interp1d()`).

Get Set Speed

Description

Returns user-provided set speed

Scheduling

Every timestep of the module (100Hz).

Pseudocode

```
float getSetSpeed()
    return this.setSpeed
```

Design Notes

N/A

Update Total Distance Travelled

Description

Calculates and updates the total distance travelled of the ego and lead vehicles in the environment.

Scheduling

Every timestep of the module (100Hz).

Pseudocode

```
void updateTotalDistanceTravelled(float egoSpeed, float leadSpeed)
```

```
        this.egoPosition += (egoSpeed * this.dt)
        this.leadPosition += (leadSpeed * this.dt)
```

Design Notes

Error may accumulate over time depending on the latency of the ego vehicle speed signal and the sample rate, but we are assuming that is negligible for the time this simulation is being run.

Get Lead Distance

Description

Gets the lead distance D_L with respect to the ego vehicle.

Scheduling

Every timestep of the module (100Hz).

Pseudocode

```
float getLeadDistance()
        return this.leadPosition – this.egoPosition
```

Design Notes

N/A

Calculate CRC

Description

Cyclic Redundancy Check calculation for data packets transmitted from environmental simulator.

Scheduling

Every timestep of the module (100Hz).

Pseudocode

```
uint8 calculateCRC(uint8* payload, int len)
        return checksum(*payload, len)
```

Design Notes

`checksum` is an abstract function that can be implemented by a library or a private function.

Calculate RC

Description

Calculates the Rolling Count of the current data

Scheduling

Every timestep of the module (100Hz).

Pseudocode

```
uint8 calculateRC(uint8 prevRC, uint8 maxRC)
        currRC = (prevRC + 1) % maxRC
        return currRC
```

Design Notes

N/A

Step Simulator

Description

Steps the simulator forward one timestep dt_{SIM}.

Scheduling

Every timestep of the module (100Hz).

Pseudocode

```
void stepSimulator()

      // advance simulated time
      this.time += this.dt

      // get ego vehicle speed from MCU; lead speed from interpolator
      float egoSpeed = deserialize(ethernetRx(IP_ADDRESS))
      float leadSpeed = this.getLeadVehicleSpeed(this.time)

      // update positions of vehicles in env. and get lead distance
      this.updateTotalDistanceTravelled(egoSpeed, leadSpeed)

      // get all outputs, calculate new rc val, serialize
      float leadDistance = this.getLeadDistance()
      float setSpeed = this.getSetSpeed()
      this.rc = this.calculateRC(this.rc, 255)
      string serializedData = serialize(leadSpeed, leadDistance, rc)

      // calculate crc from payload and append to end of message
      crcString = serialize(this.calculateCrc(serializedData,
      len(serializedData)))
      serializedData += crcString

      // transmit serialized data to MCU over Ethernet
      ethernetTx(serializedData, IP_ADDRESS)
```

Design Notes

Assuming deserialize/serialize functions are provided as a standard between subsystems (could be formats like json or protobuf or just plain ordered bytes). Also assume that we know IP address of MCU.

Uses Hierarchy

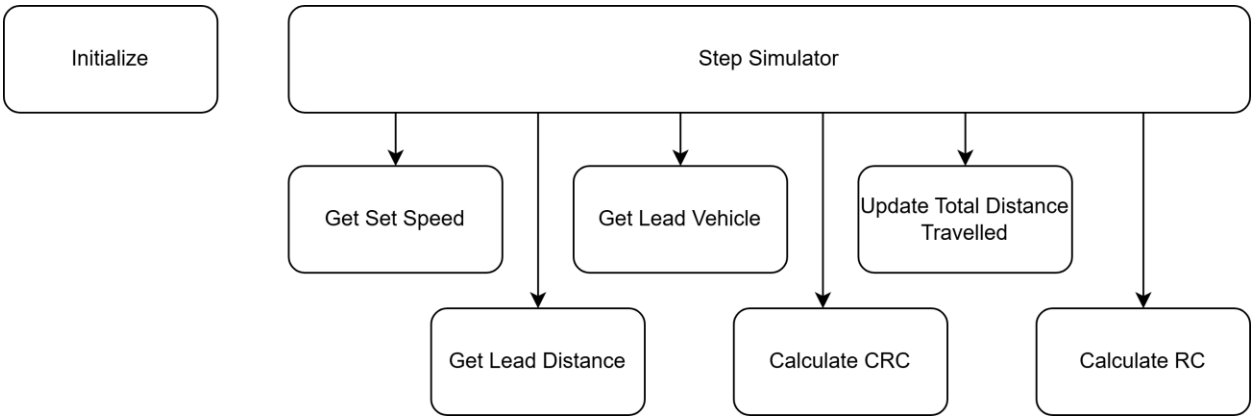


Figure 22: Uses Hierarchy for Environmental Simulator Module

M3 – Safety Monitor

Inputs

Set Speed – V_S
Lead Vehicle Distance – D_L
Lead Vehicle Speed – V_L
Ego Vehicle Speed – V_E
Cyclic Redundancy Check (Simulator and Vehicle Speed Determinator) – *CRC_SIM, CRC_ARD*
Rolling Count (Simulator and Vehicle Speed Determinator) – *RC_SIM, RC_ARD*
Vehicle Speed Fault – *Fault_Speed*

Outputs

Set Speed – V_S
Lead Vehicle Distance – D_L
Lead Vehicle Speed – V_L
Ego Vehicle Speed – V_E
State

Description

Performs various checks validating that the ACC system is in correct operation and shuts it down by setting the state to 0 if any of those checks fail. Arbitrates the actuation request from the ACC and corrects it, if necessary, before being read by the power command converter subsystem.

Hardware Mappings

Input

Name	Input Protocol/Format	Conversion	Variable Datatype
V_S	Ethernet	Zephyr Eth library	Float32

V_L	Ethernet	Zephyr Eth library	Float32
D_L	Ethernet	Zephyr Eth library	Float32
CRC_SIM	Ethernet	Zephyr Eth library	Uint8
RC_SIM	Ethernet	Zephyr Eth library	Uint8
V_E	UART	Arduino UART library	Uint8
CRC_ARD	UART	Arduino UART library	Uint8
RC_ARD	UART	Arduino UART library	Uint8
FAULT_SPEED	UART	Arduino UART library	Uint8

Output

Name	Output Protocol/Format	Conversion	Variable Datatype
V_E	Ethernet	Zephyr Eth library	Float32
Act_Req	UART	Zephyr UART library	Uint8
State	UART	Zephyr UART library	Uint8

Initialization

Activates upon powering on of system and stays active while the system is powered on.

Associated Requirements

Software Requirements	
Safety Requirements	SM_1, SM_3, SM_4
Security Requirements	
Performance Requirements	SM_2, Perf_2

Module Breakdown

Each of the Rolling Count, Cycle Redundancy and Command Range checks are separated so that in the case of an error, the source can be known. Determine State ensures all three of these checks are cleared before activating the ACC. This subsystem also saturates the actuation request from the ACC for redundancy.

Methods

Check RC

Description

Checks the rolling count signals sent from the Environmental Simulator and Vehicle Speed Determination subsystems and returns False if signals are received out of order.

Scheduling

Every time step

Pseudocode

```
bool checkRC(uint8 currRC, uint8 prevRC, uint8 RCMax)
      return (currRC == (prevRC + 1) % RCMax))
```

Design Notes

N/A

Check CRC

Description

Performs the cycle redundancy check to determine whether transmitted data (payload) is not corrupted

Scheduling

Every time step

Pseudocode

```
bool checkCRC(uint8 CRC, uint8* payload, int len)
      return (checksum(*payload,len) == CRC)
```

Design Notes

Note that the checksum logic can be hidden as we can use an external library or private function to implement it as it is a standalone computation and thus is entirely independent of the system code.

Check Command Range

Description

Verifies that the actuation request from the ACC subsystem is within in the range outlined in requirements ACC_5.1 and ACC_5.2 (either equal to zero or between 100 and 255). Returns False if this requirement is failed and True otherwise.

Scheduling

Every time step (100Hz).

Pseudocode

```
bool checkCommandRange(uint8 actReq)
      return (actReq == 0 || (actReq >= 100 && actReq <= 255))
```

Design Notes

In the future, a saturation may be added to trigger this fault only if it has been detected for multiple timesteps. If this is necessary in the final design, it will be added in the final report.

Saturate Output Command

Description

Saturates the actuation request from the ACC subsystem such that its derivative does not exceed a maximum value, as outlined in requirements ACC_5.1 and ACC_5.2.

Scheduling

Every time step (100Hz).

Pseudocode

```
uint8 saturateOutputCommand(uint8 actReq, uint8 actReqPrev)
```

```
if (actReq – actReqPrev > actReqMax)
      actReq = actReq – actReqMax
else if (actReqPrev – actReq > actReqMax)
      actReq = actReq + actReqMax

if (actReq > 255)
      actReq = 255
else if (actReq < 100 && actReq > 0)
      actReq = 100
else if (actReq < 0)
      actReq = 0

return actReq
```

Design Notes

As explained in the requirements document, actuations below 100 yield inconsistent results and may not always overcome the static friction of the wheels, so those values are prohibited. `actReqMax` is the maximum allowable change in `actReq` between time steps.

Determine State

Description

Consolidates all the other checks in the safety monitor to determine the state of ACC. If all checks pass, this will return true, indicating the ACC is in the active state, and otherwise false, indicating the inactive state.

Scheduling

Every time step (100Hz).

Pseudocode

```
bool determineState(bool crcFaultActive, bool rcFaultActive, bool
wheelSpeedFaultActive, bool outOfRangeFaultActive)
      return (crcFaultActive && rcFaultActive && wheelSpeedFaultActive &&
      outOfRangeFaultActive)
```

Design Notes

The passed parameter could be replaced by function calls for the specified checks.

Uses Hierarchy

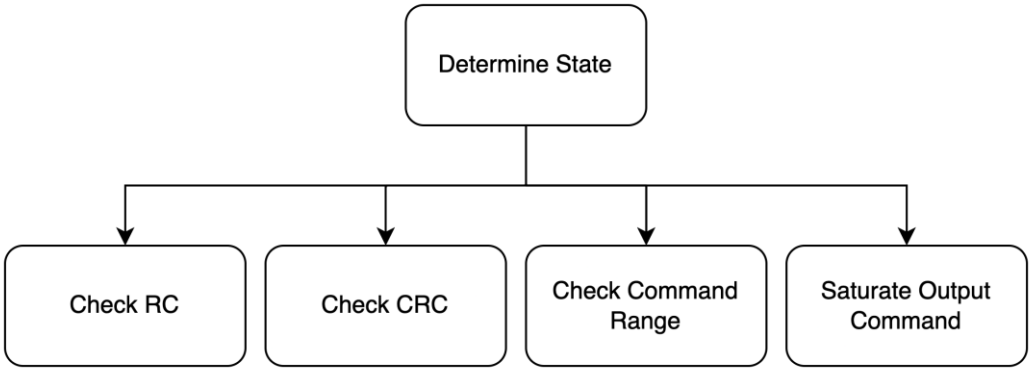


Figure 23: Uses Hierarchy for Safety Monitor Module

M4 – ACC Algorithm

Inputs

Set Speed – V_s
Lead Vehicle Distance – D_L
Lead Vehicle Speed – V_L
Ego Vehicle Speed – V_E
State

Outputs

Propulsion Command – *Act_Req*

Description

Calculates the propulsion request for the ego vehicle, based on the velocity and distance of a lead vehicle if present, or the set speed if not, using a PID controller.

Hardware Mappings

Input

Name	Input Protocol/Format	Conversion	Variable Datatype
State	UART	Zephyr UART library	Uint8

Output

Name	Output Protocol/Format	Conversion	Variable Datatype
Act_Req	UART	Zephyr UART library	Uint8
State	UART	Zephyr UART library	Uint8

Initialization

Safety monitor sets the state to active

Associated Requirements

Software Requirements	ACC_1, ACC_2
Safety Requirements	ACC_3, ACC_4, ACC_5
Security Requirements	
Performance Requirements	

Module Breakdown

This subsystem is broken down into several steps. First, the subsystem determines if a lead vehicle is present and relevant based on requirement ACC_2. This decides if the ACC should operate in cruise mode or follow mode. In cruise mode, the target speed is the set speed, while in follow mode it is the speed of the lead vehicle. A separate module calculates the actuation request based on the PID parameters and the error in velocity. Finally, this request is saturated based on ACC_5.

This subsystem uses the struct PIDParams which will have the following fields:

```
struct PIDParams {
	float P;
	float I;
	float D;
	float sum;
	float lastErr;
}
```

Because this subsystem will be coded using C, the decision was made to use a struct instead of a class.

Methods

Determine Lead Vehicle Exists

Description

Determines whether there is a lead vehicle within the simulation environment, which decides the ACC operation mode.

Scheduling

Every Time Step that State == 1

Pseudocode

```
bool determineLeadVehicleExists(float leadDistance, float leadSpeed, float
setSpeed)
	return leadDistance > 0 && leadDistance < maxLeadDist && leadSpeed <
	setSpeed
```

Design Notes

An output of true means the ACC will operate in follow mode, and false means cruise mode.

PID

Description

Initializes and implements PID control logic to calculate the actuation request on each time step.

Scheduling

`initPidParams` will be called every time state changes from 0 to 1, and pidStep will be called on every subsequent time step while state == 1.

Pseudocode

```
initPidParams(struct PidParams* params, float P, float I, float D)
      params.P = P
      params.I = I
      params.D = D
      params.sum = 0
      params.lastErr = 0

float pidStep(struct PidParams* params, float err)
      float propTerm = err * params.P

      params.sum += err
      float integralTerm = params.sum * params.I

      float derivative = (err – params.lastErr) / dtACC
      float derivativeTerm = derivative * params.D
      params.lastErr = err

      return propTerm + integralTerm + derivativeTerm
```

Design Notes

N/A

Saturate Output Command

Description

Saturates the actuation request output such that it complies with requirements ACC_3 and ACC_5.

Scheduling

Every Time Step that the State == 1

Pseudocode

```
uint8 saturateOutputCommand(uint8 actReq, uint8 actReqPrev)
      if (actReq – actReqPrev > actRateMax)
            actReq = actReq – actRateMax
      else if (actReqPrev – actReq > actRateMax)
            actReq = actReq + actRateMax

      if (actReq > 255)
```

```
        actReq = 255
    else if (actReq < 100 && actReq > 0)
        actReq = 100
    else if (actReq < 0)
        actReq = 0

    return actReq
```

Design Notes

As explained in the requirements document, actuations below 100 yield inconsistent results and may not always overcome the static friction of the wheels, so those values are prohibited. `actRateMax` is the maximum allowable change in `actReq` between time steps.

ACC Controller

Description

The main algorithm used in calculating the actuation request based on the set speed, lead distance, and lead speed. Uses a PID controller to match either the set speed or the lead speed.

Scheduling

Every Time Step that State == 1

Pseudocode

```
void initACC()
    initPID (params, P, I, D)

uint8 accController(float leadSpeed, float leadDistance, float egoSpeed,
float setSpeed)

    bool mode = determineLeadVehicleExists(leadDistance, leadSpeed,
    setSpeed)
    if (mode)
        error = leadSpeed – egoSpeed
    else
        error = setSpeed – egoSpeed

    actReq = pidStep(params, error)
    actReq = saturateOutputCommand(actReq, *actReqPrev)

    *actReqPrev = actReq
```

Design Notes

params will be a global variable. P, I, and D will be defined later when they are tuned.

Uses Hierarchy

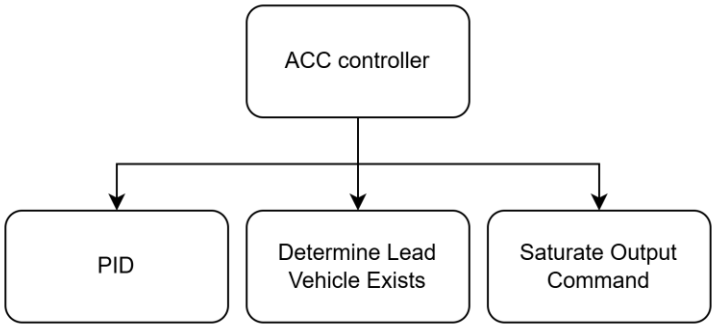


Figure 24: Uses Hierarchy for ACC Module

M5 – Power Command Converter

Inputs

Propulsion Command – *Act_Req*

Outputs

Voltage FL (V_FL)
Voltage FR (V_FR)
Voltage RL (V_RL)
Voltage RR (V_RR)

Description

Arduino code that converts the UART propulsion command signal from the ACC to a PWM command which creates torque on the wheels of the car.

Hardware Mappings

Input

Name	Input Protocol/Format	Conversion	Variable Datatype
Act_Req	UART	Arduino UART library	Uint8

Output

Name	Output Protocol/Format	Conversion	Variable Datatype
Act_Req	PWM	Arduino Analog library	Uint8
V_FL	Voltage	DRV8835 Motor Driver	N/A
V_FR	Voltage	DRV8835 Motor Driver	N/A
V_RL	Voltage	DRV8835 Motor Driver	N/A
V_RR	Voltage	DRV8835 Motor Driver	N/A

Initialization

Starts checking for inputs upon Arduino power on.

Associated Requirements

Software Requirements	ACC_1, ACC_2
Safety Requirements	ACC_3, ACC_4
Security Requirements	
Performance Requirements	Perf_3

Module Breakdown

Since this subsystem only completes one task, there is only one module.

Methods

Set Output PWM Command

Description

Converts the actuation request, which is received as a UART signal, to a PWM command to be read by the Arduino.

Scheduling

At all time steps when state == 1

Pseudocode

```
uint8 setOutputPwmCommand()
      return actReq
```

Design Notes

The software design of this system is trivial since it acts as a simple passthrough of the PWM command from the microcontroller. The real translation happens in the hardware subsystem. The `actReq` signal is read over UART and sent to two Arduino PWM analog writes. This results in two PWM signals sent to an electrical gate driver (the DRV8835) which translates the PWM command into two voltages (A and B). These two voltages are then wired in parallel to the four motors such that the voltages to both sides of the car are equal (ie. $\boldsymbol{V_{FL} = V_{RL}}$ and $\boldsymbol{V_{FR} = V_{RR}}$).

Uses Hierarchy



Figure 25: Uses Hierarchy for Power Command Converter

M6 – Vehicle Speed Speedometer

Inputs

Left Vehicle WSS – W_1
Right Vehicle WSS – W_2
Front Dyno WSS – T_1
Rear Dyno WSS – T_2

Outputs

Ego Vehicle Speed – V_E
Cyclic Redundancy Check – CRC_ARD
Rolling Count – RC_ARD
Vehicle Speed Fault – FAULT_SPEED

Description

Arbitrates between the four Wheel Speed Sensors to calculate the speed of the ego vehicle. In an ideal situation, all four sensors would agree with each other within some tolerance, and ego speed would be taken using the average of all four. If this is not the case, the wheel speed sensors take priority, with the dyno sensors as a backup. If none agree, an error is thrown and the last known speed value is used until the error is cleared or the system is shut down.

Hardware Mappings

Input

Name	Input Protocol/Format	Conversion	Variable Datatype
V_FL	PWM	Arduino Analog library	Uint8
V_FR	PWM	Arduino Analog library	Uint8
V_RL	PWM	Arduino Analog library	Uint8
V_RR	PWM	Arduino Analog library	Uint8

Output

Name	Output Protocol/Format	Conversion	Variable Datatype
V_E	UART	Arduino UART library	Float32
CRC_ARD	UART	Arduino UART library	Uint8
V_RC_ARD	UART	Arduino UART library	Uint8
FAULT_SPEED	UART	Arduino UART library	Uint8

Initialization

Starts up upon power on to Arduino.

Associated Requirements

ACC_6 was originally intended to be done within the ACC algorithm subsystem but upon further review was moved to its own subsystem.

Software Requirements	ACC_6
Safety Requirements	
Security Requirements	
Performance Requirements	

Module Breakdown

The PWM to Speed module is separated from the Calculate Ego Speed module because it is called for each WSS, while Calculate Ego Speed consolidate all of these measurements. RC

and CRC counters were separated because they are independent checks for debugging purposes.

Methods

PWM To Speed

Description

Calculates the hypothetical vehicle speed in m/s based on the PWM reading. Each time the PWM reading crosses a threshold, indicating a revolution has been completed, the ego speed is calculated from how long this revolution took to complete.

Scheduling

Every Time Step that the State == 1. This function is called for all four wheel speed sensors concurrently.

Pseudocode

```
float lastTimeStep = 0
…
float pwmToRps(uint32 currTimeStamp, uint8 currPwmVal, uint8 prevPwmVal,
uint32 prevTimeStamp)

      float speed = 0

      if (currPwmVal > PwmThres && prevPwmVal <= PwmThres)
            speed = 2*PI*radius*1000 / ((currTimeStamp – prevTimeStamp) *
      dtArd)

      if ((currTimeStamp- lastTimeStep) >= timeout)
            speed = 0

      return speed
```

Design Notes

`lastTimeStep` is given as a global variable

Calculate Ego Speed

Description

Calculates the average speed of the Ego-Vehicle based on all sensor inputs.

Scheduling

Every Time Step that the State == 1

Pseudocode

```
float egoSpeedLast = 0.0
…
bool withinRange(uint8 s1, uint8 s2, float tol)
```

```
	if (abs(s1-s2) / (s1 + s2) < tol)
		return true
	else
		return false

float calculateEgoSpeed(uint8 w1, uint8 w2, uint8 t1, uint8 t2, bool
*faultSpeed)
	bool wheelsAgree = withinRange(w1,w2,tol)
	bool treadsAgree = withinRange(t1,t2,tol)
	bool allAgree = withinRange((w1+w2)/2.0,(t1+t2)/2.0,tol)

	if ((int)(w1 > 0) + (int)(w2 > 0) + (int)(t1 > 0) + (int)(t2 > 0) <= 1)
		return 0.0
		egoSpeedLast = 0.0
		faultSpeed = false

	if (wheelsAgree && treadsAgree && allAgree)
		return (w1+w2+t1+t2)/4.0
		egoSpeedLast = (w1+w2+t1+t2)/4.0
		faultSpeed = false
	else if (wheelsAgree)
		return (w1+w2)/2.0
		egoSpeedLast = (w1+w2)/2.0
		faultSpeed = false
	else if (treadsAgree)
		return (t1+t2)/2.0
		egoSpeedLast = (t1+t2)/2.0
		faultSpeed = false
	else
		return egoSpeedLast
		faultSpeed = true
```

Design Notes

This follows the hierarchy set in requirement ACC_6. Because velocity is only updated after a full rotation, when the vehicle comes to a stop, the four sensors may detect velocity being zero at different times. To prevent an error being thrown in this scenario, if at least three sensors read zero, the ego speed will be set to zero. At this point, fault handling is not considered for rare edge cases such as a sensors constantly reading incorrect values. `egoSpeedLast` is considered to be a global variable in this implementation.

Calculate CRC

Description

Cyclic Redundancy Check for data packets received from microcontroller

Scheduling

Every Time Step that the State == 1

Pseudocode

```
uint8 calculateCRC(uint8* payload, int len)
      return checksum(*payload, len)
```

Design Notes

`checksum` is an abstract function that can be implemented by a library or a private function

Calculate RC

Description

Calculates the Rolling Count of the current data

Scheduling

Every Time Step that the State == 1

Pseudocode

```
uint8 calculateRC(uint8 prevRC, uint8 maxRC)
      currRC = (prevRC + 1) % maxRC
      return currRC
```

Design Notes

N/A

Uses Hierarchy

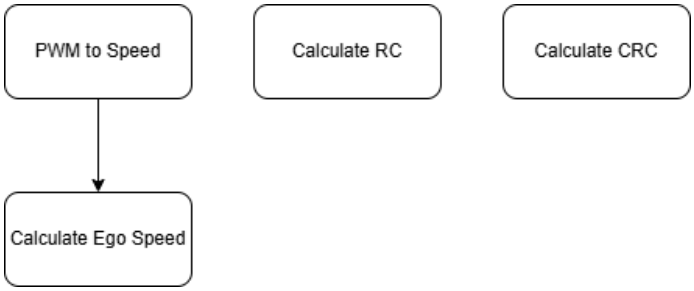


Figure 26: Uses Hierarchy for Vehicle Speed Speedometer

M7 – Vehicle Plant Model

Inputs

Propulsion Command – *Act_Req*

Outputs

Ego Vehicle Speed – V_E

Description

A software replica of the behavior of the RC car for use in Software-in-the-Loop (SIL) simulations. The module implements a mathematical model for the Ego vehicle, using a transfer

function with empirically determined constants. It can be used for comparing readings from the sensor and determining the correctness of those readings.

Initialization

Activates upon powering on of SIL simulation.

Associated Requirements

Updates ego vehicle speed at each time step. No requirements were initially written to define behavior.

Software Requirements	
Safety Requirements	
Security Requirements	
Performance Requirements	

Module Breakdown

This module is a standalone subsystem thus it does not break down any further.

Methods

Calculate Ego Speed

Description

Model of the Ego Vehicle longitudinal dynamics

Scheduling

At the same rate as the module (100Hz).

Pseudocode

```
//Global
continuousTF = transferFunction(num,den)
uint8[] previousActReq;
…
float simulate(transferFunction continuousTF, float[] t, uint8[] u)
      time, sytemResponse, stateVector = lsim(continuousTF, u, t)
      return systemResponse[len(systemResponse)-1] //last value

float calculateEgoSpeed(float dt, uint8 actReq)
      t = linspace(0,dt,tend) //timesteps for inputs
      u = previousActReq.append(actReq) // inputs
      return simulate(continuousTF, t, u)
```

Design Notes

We shall make use of external libraries such as SciPy [5], and hence hide the implementation details of creating the transfer function using `transferFunction` and simulating output using `lsim`. The transfer function coefficients (`num,den`) will be empirically determined. `tend` describes simulation end time. The empirical determination of the transfer function shall be

estimated based on the results of a swept sine command, where the input command takes a range of frequencies and the ego speed is measured. A bode plot can then be constructed from the results, and an appropriate damping ratio and natural frequency can be estimated to figure out the second order transfer function governing the main system dynamics.

Uses Hierarchy



Figure 27: Uses Hierarchy for Vehicle Plant Model Module

Normal Behaviour and Unexpected Event Handling

Normal Behaviour

The system is defined to be in normal behaviour when the *State* control variable is set to *ACTIVE.* This means that the system Safety Monitor has determined that all monitored and controlled variables are within the safe bounds defined by the requirements associated with said variables.

Unexpected Event Handling

The Safety Monitor will set the *State* control variable to *INACTIVE* in the presence of an unexpected event, this is captured as part of the else logic of the Safety Monitor design, i.e. the Safety Monitor always maintains *State* as *Active* if and only if all monitored and controlled variables are within safe bounds. This is to prevent any unexpected events to make the system accumulate errors, resulting in decreasing accuracy of the simulation.

References

[1] STMicroelectronics, "Discovery kit with STM32H735IG MCU," User Manual UM2679, [Online]. Available: https://www.st.com/resource/en/user_manual/um2679-discovery-kit-with-stm32h735ig-mcu-stmicroelectronics.pdf. Accessed: Jan. 9, 2025.

[2] Elegoo, "Elegoo Smart Robot Car Kit v4.0 Tutorial," [Online]. Available: https://www.elegoo.com/blogs/arduino-projects/elegoo-smart-robot-car-kit-v4-0-tutorial. Accessed: Jan. 9, 2025.

[3] Texas Instruments, "DRV8835 Dual Motor Driver IC," Datasheet, [Online]. Available: https://www.ti.com/lit/ds/symlink/drv8835.pdf. Accessed: Jan. 9, 2025.

[4] Texas Instruments, "DRV5057 Linear Hall Effect Sensor Datasheet," [Online]. Available: https://www.ti.com/lit/ds/symlink/drv5057.pdf. Accessed: Jan. 9, 2025.

[5] Signal processing (scipy.signal) - SciPy v1.15.0 Manual, “Signal Processing (scipy.signal)#,” [Online]. Available: https://docs.scipy.org/doc/scipy/reference/signal.html Accessed Jan. 10, 2025.